MODULE

03

Undoing things

Every mistake has an undo: restore, amend, reset, revert, and the reflog safety net.

YOU WILL LEARN

restore & amend	01_restore_and_amend.md
reset: soft, mixed, hard	02_reset_three_modes.md
revert & reflog	03_revert_and_reflog.md
Exercise: undo three mistakes	ex03_undo_mistakes/
Interview Q&A	INTERVIEW.md

WHY UNDO

Mistakes are normal. Panic is optional.

THE PROBLEM

A broken experiment in a file, a typo in a commit message, three commits that should have been one, a bad commit already on main, a branch deleted by accident...

THE IDEA

Pick the undo by asking two questions: where is the mistake (file, staging, commit)? And is it already shared?

STEP BY STEP UNDO

Which undo do I need?

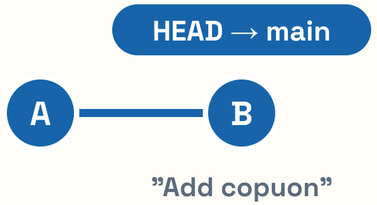
SITUATION	COMMAND
Throw away edits in a file	`git restore app.py`
Unstage a file (keep the edit)	`git restore --staged app.py`
Fix the last commit (not pushed)	`git commit --amend`
Undo local commits	`git reset --soft | --mixed | --hard HEAD~N`
Undo a commit that's already **shared**	`git revert <commit>`
"I lost commits!"	`git reflog` → reset / branch to it

> Shared history (pushed main) is never rewritten: **revert** it.

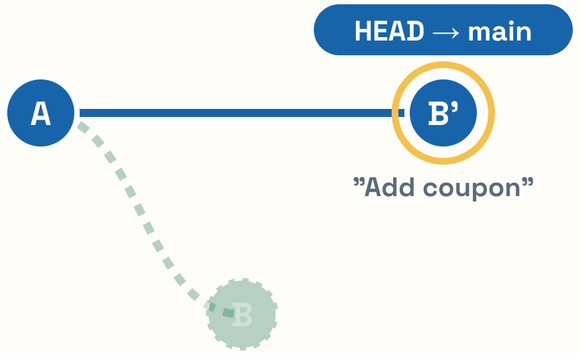
PICTURE IT AMEND

amend replaces the last commit

Before: typo in B



`git commit --amend`



B' is a brand-new commit (new ID). The old **B** is abandoned, so never amend what others already pulled.

TRY IT RESTORE AND AMEND

Fix the last commit

```
$ git add coupon.py
$ git commit -m "Add copuon"
[main 6e010f1] Add copuon
 1 file changed, 1 insertion(+)
 create mode 100644 coupon.py
$ git add test_coupon.py
$ git commit --amend -m "Add coupon with a test"
[main 1033dbe] Add coupon with a test
 Date: Tue Oct 27 08:05:00 2026 +0000
 2 files changed, 2 insertions(+)
 create mode 100644 coupon.py
 create mode 100644 test_coupon.py
$ git log --oneline --stat -1
1033dbe Add coupon with a test
 coupon.py      | 1 +
 test_coupon.py | 1 +
 2 files changed, 2 insertions(+)
```

1. Typo and a forgotten file, fixed in one step.
2. **--no-edit** keeps the old message.
3. The ID changes: only amend **unpushed** commits.

PICTURE IT RESET

reset moves the branch back

🔖 Like moving a **bookmark** back in a book. What happens to the pages after it? **--soft**: a neat pile on your desk (staged). **--mixed**: loose on the desk (unstaged). **--hard**: the shredder.

`--soft`
branch moves, changes STAGED

→

`--mixed (default)`
branch moves, changes UNSTAGED

→

`--hard`
branch moves, changes DELETED

TRY IT RESET

Same reset, three modes

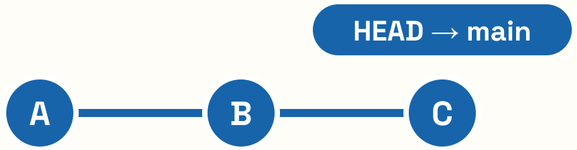
```
$ git reset --soft HEAD~2
$ git status --short
M  app.txt
$ git reset --mixed HEAD
Unstaged changes after reset:
M       app.txt
$ git status --short
 M app.txt
$ git reset --hard HEAD
HEAD is now at bcf013a Version 1
$ git status --short
$ cat app.txt
v1
```

1. After **--soft**: M in the LEFT column (staged).
2. After **--mixed**: M in the RIGHT column (unstaged).
3. After **--hard**: clean, the v3 content is gone.

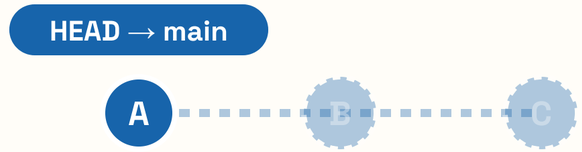
PICTURE IT RESET

reset moves the branch label back

Before



`git reset HEAD~2`



B and C are no longer on main (reflog still finds them). **--soft**, **--mixed**, **--hard** only decide what happens to their **changes**.

TRY IT REVERT

Undo a shared commit safely

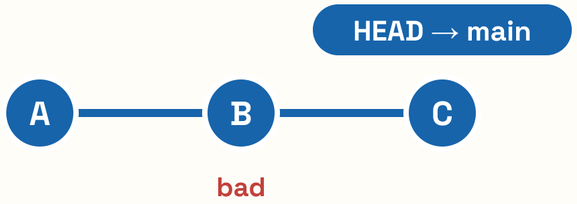
```
$ git log --oneline
62d2edc Add tax
dbce891 Change pricing
29988e9 Add shop
$ git revert --no-edit HEAD~1
[main fb0eb65] Revert "Change pricing"
 Date: Tue Oct 27 08:08:00 2026 +0000
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline
fb0eb65 Revert "Change pricing"
62d2edc Add tax
dbce891 Change pricing
29988e9 Add shop
$ cat shop.txt
price = 10
```

1. **revert** adds a NEW commit that does the opposite.
2. History stays intact: safe for main.
3. Undo a merge: **git revert -m 1 <merge>**.

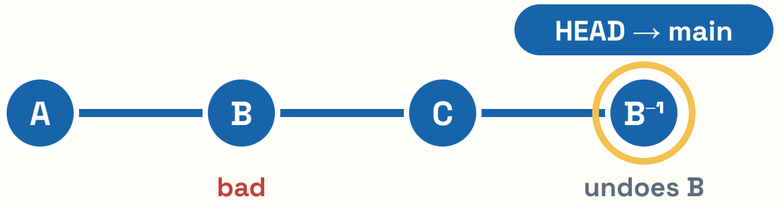
PICTURE IT REVERT

revert adds an ”undo” commit

B broke production



`git revert B`



Nothing is rewritten: history only moves **forward**. That’s why revert is safe on main.

TRY IT REFLOG

The safety net: nothing committed is lost

```
$ git reset --hard HEAD~2
HEAD is now at bcf013a Version 1
$ git log --oneline
bcf013a Version 1
$ git reflog -3
bcf013a HEAD@{0}: reset: moving to HEAD~2
1d36b1f HEAD@{1}: commit: Version 3
49ea172 HEAD@{2}: commit: Version 2
$ git reset --hard HEAD@{1}
HEAD is now at 1d36b1f Version 3
$ git log --oneline
1d36b1f Version 3
49ea172 Version 2
bcf013a Version 1
```

1. **reflog** = every place HEAD has been.
2. Find the commit, then reset or **git branch rescue <id>**.
3. Kept ~90 days, local only.

COMMON MISTAKE SHARED HISTORY

Rewriting history that others have

✗ WRONG

```
git reset --hard HEAD~1     # on main
git push --force
# teammates' next pull: chaos
```

Everyone who pulled the old commit now has history that conflicts with yours. Work gets lost or duplicated.

✓ RIGHT

```
git revert HEAD
git push
# history only moves forward
```

Rule: **reset/amend/rebase** only local commits; **revert** shared ones.

IN REAL JOBS UNDO

Undo at work

Hotfix: undo yesterday's release commit

```
git revert a1b2c3d
git push    # CI deploys the fix
```

Clean up before a pull request

```
git reset --soft origin/main
git commit -m "Add CSV export"
```

Committed to main by mistake

```
git branch feature/csv
git reset --hard origin/main
```

"I lost my work!"

```
git reflog
git branch rescue HEAD@{3}
```

INTERVIEW SAY THIS IN THE INTERVIEW

“git reset vs git revert?”

- **reset** moves the branch pointer back: history is rewritten. Use it only for **local** commits.
- **revert** creates a **new** commit that undoes an old one: history is kept, so it’s safe on shared branches.
- reset modes: **--soft** keeps changes staged, **--mixed** unstaged, **--hard** discards them.
- If you lose commits: **git reflog**.

> **Follow-up they ask next:** ”How do you undo a pushed merge?” (git revert -m 1 <merge-commit>).

RECAP REMEMBER THESE

Module 3 in 5 lines

1. **restore** files, **restore --staged** to unstage.
2. **commit --amend** fixes the last unpushed commit.
3. **reset --soft/--mixed/--hard** moves the branch back.
4. **revert** for anything already shared.
5. **reflog** finds ”lost” commits.

PRACTICE NOW

Do exercises/ex03_undo_mistakes/: three mistakes, three undos.